# CSci 402 - Operating Systems

# Midterm Exam (PM Section)

# Fall 2025

*[12:00:00pm-12:40:00pm), Wednesday, October 29)*

Instructor: Bill Cheng

Teaching Assistant: (N/A)

*( This exam is open book and open notes.*
*Remember what you have promised when you signed your*
*Academic Integrity Honor Code Pledge. )*

*( This content is protected and may not be shared, uploaded, or distributed. )*

**Time:** 40 minutes

_____-
Name (please print)

**Total:** 36 points

_____-
Signature

**Instructions**

1. This is the first page of your exam. The previous page is a title page and does not have a page number. Since this is a take-home exam, no need to sign above since you won't submit this file.
2. Read problem descriptions carefully. You may not receive any credit if you answer the wrong question. Furthermore, if a problem says *"in N words or less"*, use that as a hint that N words or less are expected in the answer (your answer can be longer if you want). Please note that points may get *deducted* if you put in wrong stuff in your answer.
3. If a question doesn't say `weenix`, please do not give `weenix`-specific answers.
4. Write answers to all problems in the **answers text file**.
5. For non-multiple-choice and non-fill-in-the blank questions, please show all work (if applicable and appropriate). If you cannot finish a problem, your written work may help us to give you partial credit (if applicable). We may not give full credit for answers only (unless you are asked to give only answers). Grading can only be based on what you wrote and cannot be based on what's on your mind when you wrote your answers.
6. Please do *not* just draw pictures to answer questions (unless you are specifically asked to draw pictures). Pictures will not be considered for grading unless they are clearly explained with words, equations, and/or formulas. It's very difficult to draw pictures in a text file and you are not permitted to submit additional files other than the answers text file.
7. For problems that have multiple parts, please clearly *label* which part you are providing answers for.
8. Please ignore minor spelling and grammatical errors. They do not make an answer invalid or incorrect.
9. During the exam, please only ask questions to *clarify* problems. Questions such as "would it be okay if I answer it this way" will not be answered (unless it can be answered to the whole class). Also, you are suppose to know the definitions and abbreviations/acronyms of *all technical terms*. We cannot "clarify" them for you. We also will **not** answer any clarification-type question for multiple choice problems since that would often give answers away.
10. Unless otherwise specified and stated explicitly, multiple choice questions have one or more correct answers. You will get points for selecting correct ones and you will lose points for selecting wrong ones.
11. When we grade your exam, we must assume that you wrote what you meant and you meant what you wrote. So, please write your answers accordingly.

(Q1) (2 points) Let's say that you have an infinitely fast and accurate computer and you run your warmup2 on it with the following commandline: "`./warmup2 -r 1000 -t g0.txt`" and the content of `g0.txt` is as follows:

```
4
4    4    5
5    5    14
3    5    6
3    2    1
```

How many **milliseconds** into the simulation will packet `p4` (i.e., the 4th packet) leaves the system? Please just give an integer value answer (no partial credit for this problem).

(Q2) (2 points) Let say that X, Y, Z are regular Unix user processes with PIDs 14, 23, and 7, respectively. Furthermore, Z's parent is Y and Y's parent is X. If process Y dies while processes X and Z are still running, what would happen?

(1) process X becomes process Z's new parent
(2) the OS kernel will terminate process Z because its parent is now dead
(3) the INIT process becomes the new parent of both processes Y and Z
(4) process Z becomes parentless
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q3) (2 points) Which of the following statements are correct about a newly created thread?

(1) it will be in the "waiting" state to wait for a CPU to become available
(2) its state will be uninitialized (i.e., in a random state)
(3) it will be in the same state as the process it's in
(4) it will copy the state from its parent thread
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q4) (2 points) For an **opened file** in Unix, what **context information** is stored in a **file object** in the kernel's system file table?

(1) file type of the opened file
(2) access mode (i.e., access permissions of the file)
(3) a pointer that points to the directory that contains this file
(4) user ID and group ID of the opened file
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q5) (2 points) On Unix/Linux systems, **sequential I/O** on regular files is done by calling **read/write()** system calls. Which of the following statements are true about Unix/Linux **block I/O** on regular files?

(1) to perform block I/O, you need to open a file in the block I/O mode, then use read() and write() system calls to perform block I/O
(2) to perform block I/O, you need to map a file (or part of a file) into your address space
(3) using the write() system call can also be considered as doing block I/O if the last argument of write() is the size of a disk block
(4) block I/O is only available to kernel processes and not available to user processes
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q6) (2 points) Let **cv** be a POSIX condition variable and **m** be a POSIX mutex. When thread X calls **pthread_cond_wait(cv,m)**, some operations are **locked** together within one **atomic operation**. Which of the following operations are part of this atomic operation?

(1) unlock the cv
(2) thread X adds itself to the cv queue
(3) thread X adds itself to mutex m's queue
(4) lock mutex m
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q7) (2 points) The code below is the body of a function called barrier_sync(), which was suggested as a "solution" to the **barrier synchronization problem** (to synchronize `n` threads) and it's implemented using a POSIX mutex **m** and a POSIX condition variable **BarrierQueue**.

```
int count = 0;
void barrier_sync() {
  pthread_mutex_lock(&m);
  if (++count < n) {
    while (count < n)
      pthread_cond_wait(&BarrierQueue, &m);
  } else {
    /* release all n-1 blocked threads */
    pthread_cond_broadcast(&BarrierQueue);
    count = 0;
  }
  pthread_mutex_unlock(&m);
}
```

Assuming that all the variables have been properly initialized, which statements below are correct about the above code if it's used to synchronize first $n$ threads that call barrier_sync()?

(1) the first thread that enteres the barrier will be the first thread that leaves the barrier
(2) only the $n^{th}$ thread will leave the barrier while the other $n-1$ threads will get stuck at the barrier
(3) out of the first $n$ threads, the $n^{th}$ thread will be the first thread that leaves the barrier
(4) some threads may leave the barrier before the $n^{th}$ thread arrives
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q8) (2 points) What are the consequences and implications of the phrase "the kernel is very very powerful and can do anything it wants" as far as our entire **weenix** Kernel Assignment 1 is concerned?

(1) without QEMU, you can still run weenix
(2) if the kernel deadlocks when the grader is grading your work, it's due to bugs in your code
(3) if the buddy system runs out of memory, it's due to bugs in your code
(4) if the run queue is empty, it's due to bugs in your code
(5) if you get a kernel panic, it's due to bugs in your code

Answer (just give numbers): _____

(Q9) (2 points) Which of the following operations are **locked** together in an **atomic operation** when **sigwait(set,...)** is called (where **set** specifies a set of signals)?

(1) it clears all pending signal specified in **set**
(2) it unblocked the signals specified in **set**
(3) it waits for any signal specified in **set** to occur
(4) if a signal specified in **set** is generated, the corresponding signal handler is called
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q10) (2 points) Which statements are correct about **Unix signals**?

(1) an application can ask the OS to not deliver some signals
(2) when a signal is generated, if it's blocked, it is discarded (i.e., as if it was never generated)
(3) by convention, a signal handler should return a value of zero if the signal was handled successfully and return a non-zero value otherwise
(4) when a signal is generated, if it's blocked, the kernel will get it delivered by creating a new user thread
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q11) (2 points) Which of the following statements are correct about **weenix**?

(1) **weenix** kernel uses scheduler functions such as sched_wakeup_on() and sched_broadcast_on() to give the CPU from one kernel thread to another
(2) for your kernel 1 to work correctly, it's sufficient that the **weenix** scheduler is a simple sequential (e.g., first-in-first-out) scheduler
(3) in **weenix**, the scheduler decides which thread gets cancelled and which thread gets terminated
(4) when a kernel thread in **weenix** "goes to sleep" (i.e., give up the CPU and the thread is not dead), it must "sleep" in a queue somewhere
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q12) (2 points) In the most common OS implementation, when a user thread makes a system call, it simply becomes a kernel thread. In such an implementation, how is this kernel thread different from the original user thread (assuming that you are running on an x86 CPU)?

(1) user thread runs with interrupt disabled while kernel thread runs with interrupt enabled
(2) they access different parts of the physical memory
(3) their text segments are in different virtual address ranges
(4) they use different CPU registers to point to their respective stack segments
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q13) (2 points) Which of the following statements are correct about a **Unix file system**?

(1) in the Unix file system hierarchy, it's possible that a node in the hierarchy has more than one "parent" node
(2) an Unix file system is a strict "tree hierarchy"
(3) a directory can be contained in multiple "parent" directories
(4) symbolic links to the same file cannot be contained in multiple directories
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q14) (2 points) What does **async-signal safety** mean?

(1) one should always deal with signals synchronously
(2) make sure your code is safe when dealing with asynchronous signals
(3) make sure that your code that handles an asynchronous signal won't cause deadlocks
(4) do as little as possible inside a signal handler
(5) if you use a global variable inside a signal handler, your code will not be safe

Answer (just give numbers): _____

(Q15) (2 points) In a multi-threaded process, if a **detached** POSIX thread calls pthread_exit(), it cannot delete its user-space stack inside pthread_exit(). Which of the following statements are correct about when and/or where the stack of this **detached** thread get freed up (i.e., destroyed)?

(1) only the kernel can perform such a task
(2) a "reaper" thread can be used to perform such a task
(3) such a task can only be performed only when another thread in the process calls exit()
(4) such a task can only be performed inside a signal handler
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q16) (2 points) What was involved in POSIX's **solution** to provide **thread safety** for accessing the global variable **errno** when a system call returns?

(1) when **errno** is accessed, a software interrupt is generated so that the kernel can provide thread safety
(2) define **errno** to be a macro/function call that takes the PID of the calling process as an argument
(3) when **errno** is accessed, a segmentation fault is generated so that the kernel can provide thread safety
(4) use thread-specific **errno** stored inside the thread control block
(5) none of the above is a correct answer

Answer (just give numbers): _____

(Q17) (2 points) Let's say that your **umask** is set to **0614**. (a) What file permissions will you get (in octal) if you use an editor to create the **warmup1.c** file? (b) What file permissions will you get (in octal) if you use a compiler to create the **warmup1** executable? Please note that if your answer is not in octal, you will not get any credit.

(Q18) (2 points) Assuming that you only have one processor, which of the following statements are correct about **interrupts**, **traps**, and **Unix signals**?

(1) an interrupt is most likely caused by the currently running thread
(2) even when a process is in the zombie state, its thread can still cause a trap
(3) a signal can be blocked/masked while an interrupt cannot be blocked/masked
(4) for all signal types, a user program can catch that signal by specifying a user space signal handler
(5) none of the above is a correct answer

Answer (just give numbers): _____